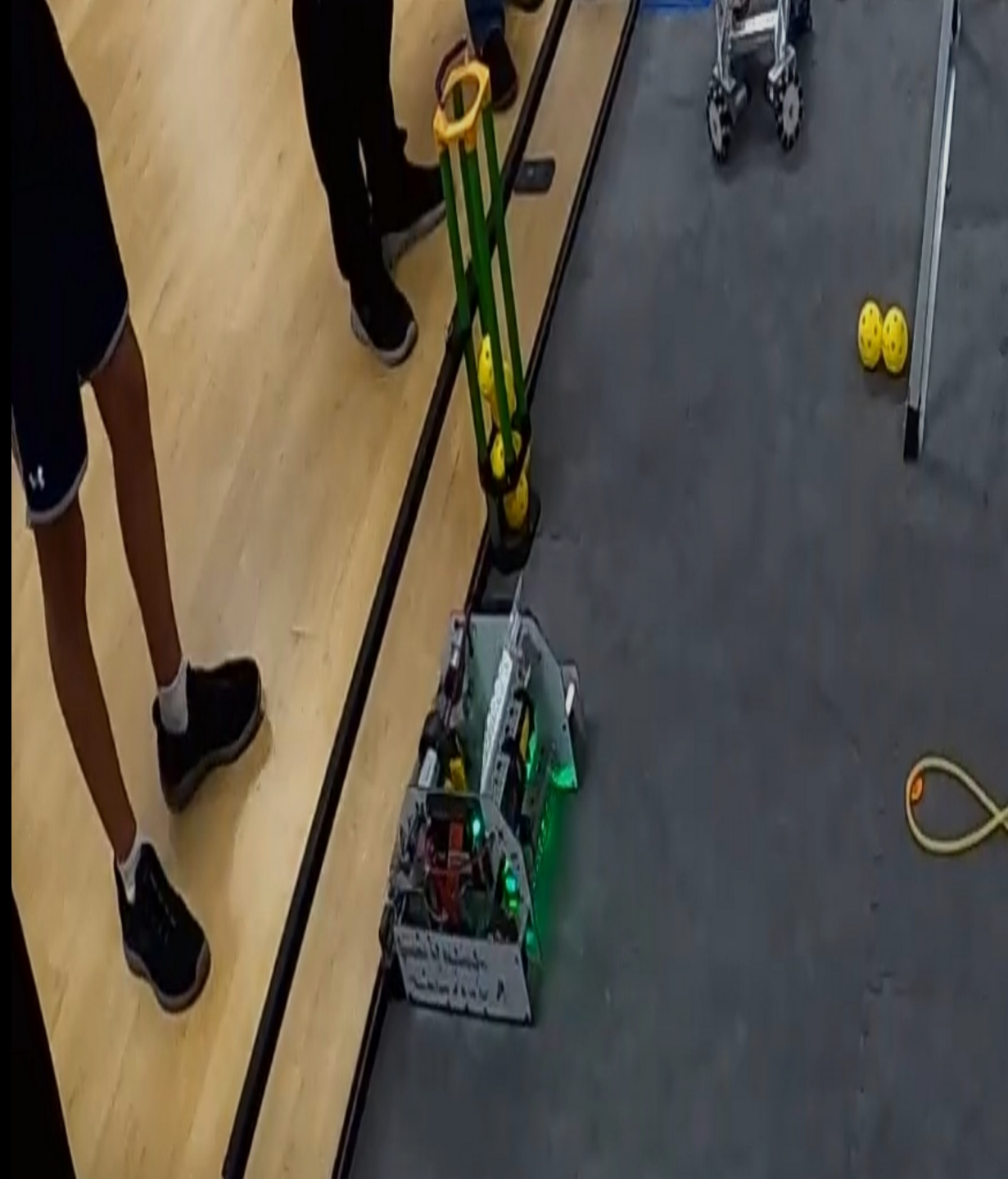




Describe your robot. Let AI write the code. Test. Repeat.

What The Hive learned watching our mentors code a robot in one week with Gemini in Android Studio — and how your team can do it too.

The Hive • FTC 3747 • Beehive Academy



Who we are, and what this is

1

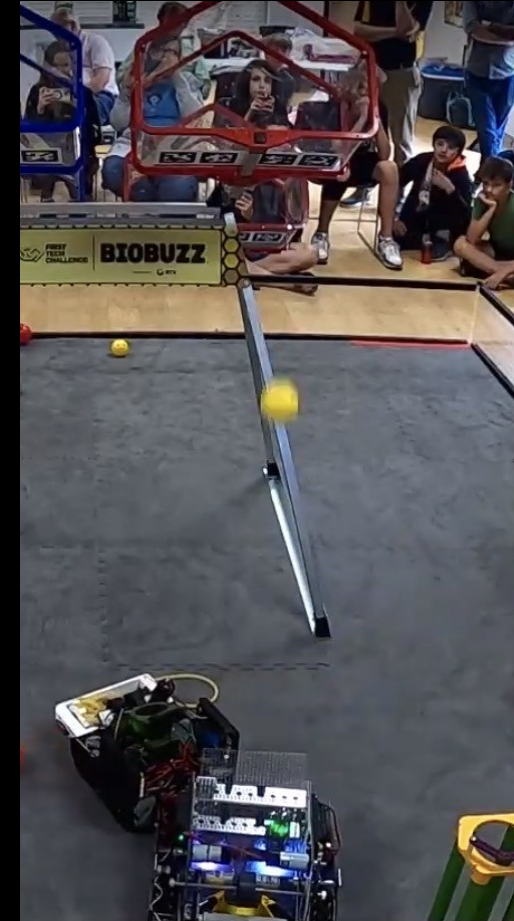
We're The Hive, FTC 3747. This year our students built and coded our robot by hand. No AI.

2

Our mentors entered the mentor competition with a separate robot, and ran an experiment on it: no hand-written Java. Everything described to Gemini.

3

We watched both. This is what the experiment showed, what we'd do with it, and how you can use it.



The mentor robot in a match



The problem every team has



**Most teams have 1
programmer.**

Sometimes 0.



**Every mechanical change
waits**

on that one person.



**The mechanical people
know what the robot
should do.**

They just can't type it in Java.



The programmer doesn't go away. The job changes.

Before anyone else touches it

Describe the robot — every motor and servo, name, port, direction.

Architect it — subsystems, state machines, what the autonomous will need later.

Set the process — one change per prompt, test, commit.

That groundwork is what lets a mechanical teammate make a change in English and have it land in the right place.

After every change

Review what the AI did — and catch drift.

On the mentor robot: the autonomous that drove the intake through a fake gamepad. The D-pad that ended up doing two things — slow mode and flywheel tuning — instead of one.

Both worked. Both were wrong. Only a programmer reading the code saw it.

The programmer gets the big picture back: “What do I want vision to do?” instead of “How do I parse the Limelight result?”



AI can shrink your skills. Or multiply them.

-17%

on the exam

PNAS 2025 · about 1,000 high-school students (math), randomized. Plain ChatGPT: practice +48%, unassisted exam -17%. A hints-only tutor “largely mitigated” the harm.

+19%

longer, not shorter

METR 2025 · 16 expert maintainers on their own code, 246 tasks. With AI they took 19% longer — and believed they'd been 20% faster.

50 vs 67

comprehension score

Anthropic, Jan 2026 · their own tool, their own study (not peer-reviewed). 52 mostly-junior engineers: AI group understood less, finished ~2 min faster. Delegating hurt; asking questions didn't.

The difference is how you use it. Delegate and auto-accept: you lose understanding. Ask, read the diff, explain it: you don't. That's our rule — next slide.



Our rule: no AI until you can debug without it

“Do not use AI to help you on these assignments. We will probably use AI during the season. However, if you do not understand the basics of programming taught here, you will not be able to debug the code during the season. And, it becomes painfully obvious if you use AI because you cannot answer how your program works.”

— The Hive, 2026 Programming Training Checklist

14

chapters of Learn Java for FTC, by hand — read, type, do the exercise

40

FTCSIM exercises — Intro, Movement, Sensors, Puzzles, Grabby

0

AI until the mentor has checked every box, in order

then

AI as a season tool — and you still explain the code

Same rule as AP Computer Science Principles: “students must be prepared to explain their code in detail.” (College Board) Same rule as FTC judging: students answer; adults may only observe.

Fundamentals first, then AI. We think that's how you get the speed without losing the understanding.



Seventy years of “that's not real programming”

1954	FORTRAN	<i>“the strength of the skepticism about ‘automatic programming’... as it existed in 1954” — John Backus, who led the FORTRAN team, writing in 1978</i>
1983	Pascal	<i>“Real Programmers use FORTRAN. Quiche Eaters use PASCAL.” — Ed Post</i>
2005	IDE autocomplete	<i>“And I think it's making us dumber.” — Charles Petzold, on IntelliSense</i>
2017	Stack Overflow	15.4% of 1.3 M Android apps contained security-related code copied from Stack Overflow; 97.9% of those had an insecure snippet — Fischer et al., IEEE S&P
2026	AI coding	Gemini drove our mentors' autonomous through a fake gamepad. It worked. A human reading the code caught it.

Every layer took the typing. None took the architecture, the testing, or the reading. — “the hard part of building software [is] the specification, design, and testing of this conceptual construct, not the labor of representing it.” Fred Brooks, 1987



What the mentors did — the week

- Sep 17, 10:26 PM** Ben (coordinator) describes the whole robot → working TeleOp that night, subsystem classes (25 minutes, 5 prompts)
- Sep 18, ~9:50 PM** First autonomous state machines, Shoot First / Shoot Delayed — Asim's design, called out at the robot, typed by Ben
- Sep 19, 8:29 AM** Asim (mechanical), at the keyboard: “holding both joysticks left made the robot strafe right, and vice versa. please fix”
- 8:36 AM** “it is spitting balls out, so i would like to shorten it” — reverse bump 200 → 100 ms
- 8:40 AM** “we get points for not touching the wall. Can we add a backup after the turn and drive forward?”
- 11:35 AM** “I thought gamepad 1 dpad was slow mode, not tuning the flywheel.”
- 11:37 AM** “I don't want the intake to resume. I want the intake to stop (whether it was stopped or running).”
- 2:57 PM** “Let's set the default speed to 0.95” — default changed from 1.0
- Scrimmage** Autonomous scored; TeleOp drove.

The screenshot shows an IDE with several Java files open: `MecanumTeleOp.java`, `Flywheel.java`, `Intake.java`, and `Flywheel.java`. The `MecanumTeleOp.java` file is the active editor, showing a class that extends `OpMode` and implements `RobotController`. It includes comments and code for drive motors, flywheel, intake, and LED control. To the right of the IDE is a chat window titled "Agent" with the prompt "Create FTC Mecanum OpMode with flywheel and intake controls." The chat shows a user prompt: "what is the current time of spinning the motors back before starting the flywheel? It is spitting balls out, so i would like to shorten it". The AI response includes "Thought" and "The current settings for reversing the motors before starting the flywheel are:" followed by a list of parameters: "1. In TeleOp (Flywheel.java): Reverse Duration: 200 ms (0.200 seconds), Reverse Power: -0.5, Pause Duration (before spooling forward): 300 ms (0.300 seconds); 2. In Experimental Auto (BaseAutoExperimental.java): Reverse Duration (4th Ball): 300 ms (0.300 seconds), Reverse Power: -0.5". Below this, it lists "Recommended Options to Prevent Spitting Balls Out:" with two suggestions: "1. Shorten Duration: Reduce the reverse bump time from 200 ms down to 100 ms (or 75 ms). 2. Reduce Reverse Power: Reduce the reverse power from -0.5 down to a gentler -0.2 or -0.25 so it gently pulls balls back without ejecting them out the front." It also asks "What duration (e.g. 100 ms) and power level (e.g. -0.2) would you like to set for both TeleOp and Autonomous?" and mentions "Generated by gemini-3.6-flash". At the bottom of the chat window, it says "Changes (2 artifacts)" and "Ask AI".

Sep 19, 8:36 AM, as typed — and Gemini's reply: it reports the numbers (200 ms at -0.5, 300 ms pause), offers two options, then changes both at once. The next prompt put the power back.

67 prompts, two days and an evening. 0 lines of Java typed.



A yellow chevron graphic pointing to the right, located on the left side of the slide.

“I've written Java for twenty-five years. I didn't type a line of it. After the first build, the changes came from the mechanical mentors — sometimes typed by them, sometimes called out from over the robot while I typed. They don't write Java.”

— Ben, our coordinator

Hand-coding vs. architecting with AI

Our robot — hand-coded by us

3 of us could code it: Tom, Sophi, Sadiqah. Siloed — Tom wrote TeleOp, Sophi wrote autonomous

4 days of coding

A borrowed sample autonomous only powered two wheels — the other two were braking. Tom saw the robot crawl in morning testing, then had to read Sophi's code cold to find why

Programmer codes while the mechanical teammate waits; often one programmer does it all and kludges around mechanical problems. One new autonomous state: 10–20 minutes of boilerplate

Each of us understood our own file. Nobody could summarize the other's

Mentor robot — described to Gemini

2 people wrote prompts — one writes Java, one doesn't

67 prompts over two days and an evening; working TeleOp the first night

Mechanical mentor fixed strafe, flywheel bump and controls — in English, no Java

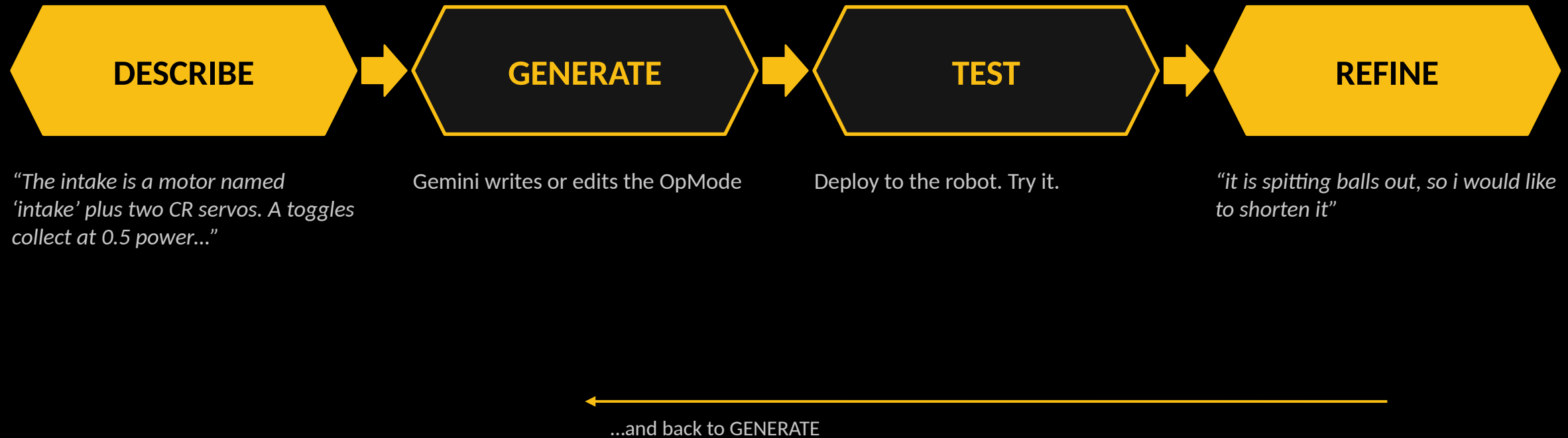
Firewheel transfer servos came out Sep 18 (reported 7:50 PM); code reworked that night by describing the new design

Someone still has to read what Gemini wrote — the Reader role

The bottleneck moved: from “who can type Java” to “who can describe the robot precisely.”



The method: describe → generate → test → refine



The skill you build is **DESCRIBE** and **REFINE**: names, directions, timing, order.



The checkpoints — one trip around the loop each

- | | | | |
|---|--|--|--|
|  0 | Install Android Studio + FtcRobotController, sign in to Gemini |  5 | Refine: the mechanical mentor's four real prompts |
|  1 | Describe the robot: names, ports, purposes, directions |  6 | Auto v1: back off the wall and shoot → test |
|  2 | Mecanum drive → test |  7 | Auto v2: Shoot First / Shoot Delayed, which parks → test |
|  3 | Intake → test |  8 | Auto v3: under 30 seconds, keep the LEAVE points → test |
|  4 | Shooter: flywheel startup sequence + pulsed feed → test |  9 | Extras: LEDs, endgame rumble, the 4-ball experiment |

github.com/The-Hive-3747/Teaching-AI-Coding — a guide page and a video per checkpoint



What a good description has

1

Names

exactly as in the Robot Controller config: `front_left_drive`, not “the front left motor”

2

Purpose

what the part does for the robot

3

Direction

which way is “forward”; which motors are mounted mirrored

4

Control

which button or stick; toggle or hold; what happens on release

5

Timing and order

what has to happen before what, and for how long

Anything you leave out, Gemini guesses.



Vague vs. specific

Vague

“Add a shooter.”

Which motor? Which way? Which button? Toggle or hold? What happens first?

Specific

“One motor named flywheel, reversed. Pressing B runs the flywheel and intake backward at -0.5 for 200 ms to clear a jammed ball, stops everything for 300 ms, then runs the flywheel forward at full power and keeps it there. Pressing B again stops it. While it's running and right bumper is held, pulse the intake 100 ms on at 0.5, 200 ms off, so balls feed one at a time.”

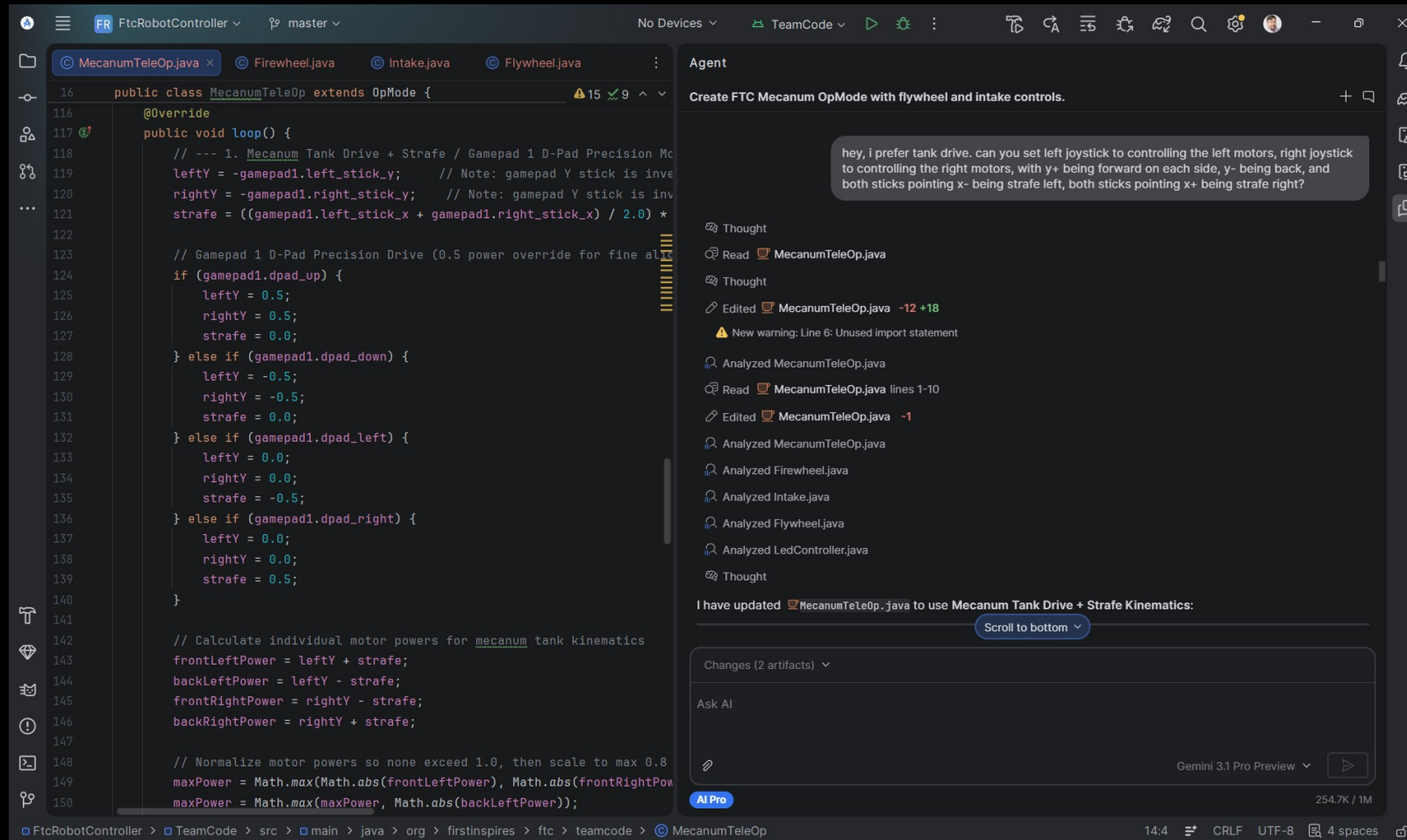
Real — Sep 19 session, verbatim

“When I start up the flywheel, I want the process to continue to do the reverse function. But then I don't want the intake to resume. I want the intake to stop (whether it was stopped or running). Does that make sense?”

No code words. A sequence. A few lines changed in Intake.java.



Now watch it happen



The screenshot shows an IDE with a Java file named `MecanumTeleOp.java` open. The code implements a tank drive system for a robot. The `loop()` method reads joystick inputs and calculates motor powers for four motors (front-left, back-left, front-right, back-right) based on the joystick's X and Y coordinates. The code includes comments explaining the kinematics and a warning about an unused import statement.

On the right side of the IDE, there is an AI chat interface. The prompt is: "Create FTC Mecanum OpMode with flywheel and intake controls." The AI's response is: "hey, i prefer tank drive. can you set left joystick to controlling the left motors, right joystick to controlling the right motors, with y+ being forward on each side, y- being back, and both sticks pointing x- being strafe left, both sticks pointing x+ being strafe right?" The chat interface also shows a list of files analyzed by the AI, including `MecanumTeleOp.java`, `Firewheel.java`, `Intake.java`, and `Flywheel.java`.

The mentors' tank-drive prompt, as typed, and the code it produced. You're about to see the same thing live.

- Description already sent
- Drive prompt, live
- Read the code: names match
- Build, deploy, drive
- Refine: the real strafe fix
- Deploy, drive again



Where it goes wrong — and the fix

Symptom	Cause	Fix
Won't compile: "cannot find symbol"	Gemini invented a name	Paste the error back: "Fix this compile error"
Strafe is mirrored	Mecanum mixing signs	<i>"holding both joysticks left made the robot strafe right, and vice versa. please fix"</i>
Flywheel startup spits balls out	Reverse bump too long	<i>"what is the current time of spinning the motors back...? it is spitting balls out, so i would like to shorten it"</i>
Two controls on one button	Control scheme drifted	<i>"I thought gamepad 1 dpad was slow mode, not tuning the flywheel."</i>
Auto drives the intake through a fake gamepad	Nobody told Gemini an autonomous was coming	Say it up front: "An autonomous will use this class later — put the behavior in plain methods." Gemini takes the expedient route unless you say what's coming.
Auto runs over 30 s	Gemini was told the limit; nobody saw the sum	"Add up every state's duration and tell me the worst-case run time." Then cut one number.
Gemini rewrote everything, broke what worked	Didn't say "only change X"	"Only change the intake section. Leave the drive code as is."

When it's wrong, the description was missing something. Say the missing thing.



Lehi, 2026: it's in the job posting

Podium • Lehi, Utah • Software Engineering Intern

"Familiarity with AI-assisted development tools such as Cursor, Claude Code, Codex, GitHub Copilot, or similar." — alongside "Strong programming fundamentals..."

Podium • Lehi, Utah • Senior Software Engineer

use AI tooling "as a genuine force multiplier, not a novelty" • "A hands-on, daily relationship with AI coding tools"

Affirm • US fintech (remote role, Poland)

"every Affirm engineer builds with tools like Claude Code and most PRs are co-authored with AI... excelling in the reviewer's seat... is a core expectation"

1Password • Rust Developer

"Effective at using AI tooling to accelerate development, testing, debugging..." — and the interview is run without AI

2.5%

of all US job postings asked for AI skills in 2025 — up 55% in one year (Stanford AI Index 2026 / Lightcast)

10%

of 4,820 software-engineering postings required AI coding tools in one Sep 2026 scan; 20% mentioned them; interns highest at 27% (match.dev)

"baseline"

"Reflexive AI usage is now a baseline expectation at Shopify."
— Tobi Lütke, CEO, Apr 2025 (as quoted in press)

Job ads that ask for AI skills advertise higher pay — PwC 62% within occupation (global, 2025), Lightcast 28% (US, 2024). Vendor data; correlation, not a promise.



The door is narrower. The tool is expected.

–20%

Software developers aged 22–25: employment down nearly 20%, late 2022 → Sep 2025 (payroll data). For young workers in AI-exposed jobs the gap widened to 19% by Aug 2026. — Stanford “Canaries in the Coal Mine”

16.5%

of entry-level job descriptions now emphasize AI skills, up from 10.5% in fall 2025; employers say over a third of entry-level jobs require them. — NACE, spring 2026 (185 employers)

50.5%

of graduating seniors aren't building AI skills at all. — NACE student survey, 2026

“The value of 90% of my skills just dropped to \$0. The leverage for the remaining 10% went up 1000x.”

— Kent Beck, creator of Extreme Programming, April 2023

We think: the jobs disappearing are the ones AI does instead of a person. The ones being posted are the reviewer's seat. The checklist teaches the 10%.



Running this with your team

Describer

Whoever knows the robot best says what to change. They can type it or someone else can — what matters is whose words they are.

Tester

Deploys and drives. Says exactly what happened.

Reader

Reads the generated code aloud and explains it. This is where the learning happens.

Coach

Asks “what did you tell it?” when something breaks. Doesn't type.

Ground rules

- One change per prompt. Test after every change.
- Commit (or copy the file) after every working test.
- If Gemini rewrites something that worked, say “only change X.”
- The Reader has to be able to explain the code before the team moves on.



What you need to start



A laptop that runs Android Studio (Windows, Mac or Linux)



A Google account signed in to Gemini in Android Studio



The FtcRobotController project cloned



Your robot's hardware config done on the Driver Hub — names and ports



A written list of every motor and servo: name, port, purpose, direction



The guide and videos: github.com/The-Hive-3747/Teaching-AI-Coding

Or bring your robot to a Hive training session. We'll teach your students to prompt. You leave with a driving robot — maybe more.



First Gradle sync — three pop-ups

Accept the Daemon JVM toolchain migration

Decline the Gradle / AGP upgrade — it breaks the FTC project

Accept the Windows Defender exclusion, or builds crawl

Checkpoint 0 has all three.

Who made this

Tom, Sophi and Sadiqah — The Hive's programmers

Hand-coded the team's own robot this season. Present this workshop and narrate the videos.

Ben — coordinator

Described the mentor robot's architecture to Gemini on night one. Typed no Java.

Annie — head coach

Asim — mechanical mentor

Built the mentor robot; authored most of the later changes in plain English, eight of them at the keyboard.

Made the way it teaches: described by the team, generated, tested, refined.

Royd — coach and mechanical engineer

Second coach on The Hive.

Claude (Anthropic)

Compiled the transcript, guide and slides from the team's material.

Gemini in Android Studio (Google)

Wrote every line of the mentor robot's code from the mentors' descriptions.



Start small.

Test every step.

When it's wrong, describe what was missing.

github.com/The-Hive-3747/Teaching-AI-Coding

Guide, videos, every real prompt, and the code — all of it.

Training sessions: bring your robot, leave with it driving. Find us at the pits: The Hive, FTC 3747.



**The Hive
3747**